# Project Report: *Quill*



| Name | Student ID |
|---|---|
| Sudarshan Bashyal | 1751835 |
| Avnish Raut | 1751366 |



## 1. Use Case

- **Problem statement:** Students accumulate notes, voice memos, sketches and flashcards across multiple apps (a notes app, a separate whiteboard/drawing tool, a flashcard app, sometimes a voice recorder) with no single place that ties them together and with no offline-first guarantee when they need to review on the go or in a lecture hall with poor connectivity.
- **Motivation:** Quill was built to consolidate note-taking, hand-drawn whiteboards, voice memos, spaced-repetition flashcards and quizzes into one offline-first Android app, so a student's study material lives in one place instead of being scattered across single-purpose tools.
- **Real-world relevance:** Every student who studies from lecture notes has hit the same friction: sketching a diagram means switching apps, turning notes into flashcards means re-typing them elsewhere and studying on a commute or in a room with no signal means half of these tools stop working. Quill's offline-first design (local SQLite, no server, P2P collaboration over Nearby Connections instead of the internet) targets exactly that gap.

## 2. UX Design and Target Group

### 2.1. Target Group

- **Primary users:** (who, specifically — not “everyone”)
  University/college students who take notes during lectures or self-study and want to turn those notes into review material (flashcards, quizzes) without switching apps.

- **Secondary users:** (user groups that interact with the app but aren’t the primary target)
  Study-group partners who join a live collaborative whiteboard session with a primary user (e.g. to co-annotate a diagram during a group study session) and users of the Wear OS companion (same primary user, on their watch) who capture a quick voice memo or review due flashcards without pulling out their phone.

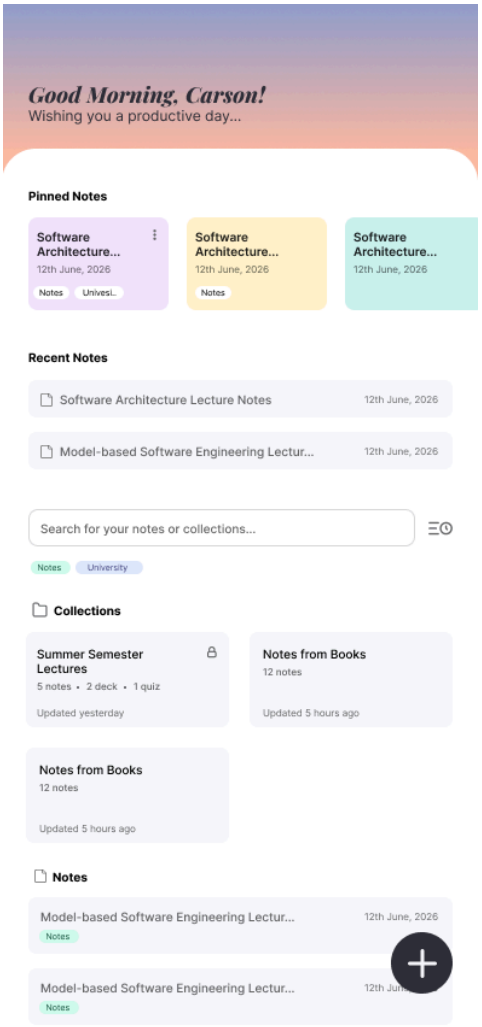
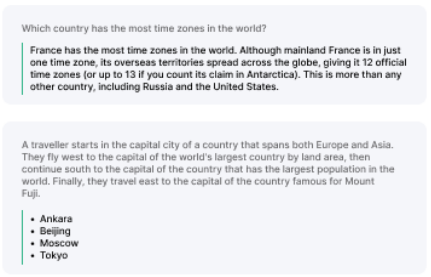
- **User needs:** (what problem are they trying to solve, what’s their current workaround?)
  Capture information quickly during a lecture (typed notes, voice memo, or a quick sketch) in a single tool; convert that material into spaced-repetition flashcards and quizzes; keep

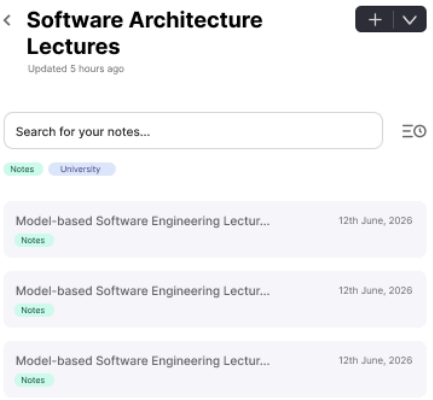
sensitive collections (e.g. exam drafts) locked behind biometrics; study on the move, including from a smartwatch, without needing network connectivity.

- **Key usage scenario / user story:** (1 short narrative: who, where, why they open the app)
  A student is in a lecture and the professor draws a diagram on the board. Rather than switching to a separate whiteboard app, they open Quill, sketch the diagram directly into their lecture note, and later convert the key definitions in that note into flashcards for spaced review, also reviewing the first few due cards from their watch on the walk out of the building.

## 2.2.Mockups

- Include at least 2-3 key workflows (low-fidelity is fine: hand-drawn/scanned, Figma, Excalidraw, or AI-generated – tool is your choice, just report what you used).
- For each mockup, add one sentence: which user needs (from 2.1) does this screen/workflow address?
- If AI was used to generate or refine mockups, name the tool here (detailed reflection goes in Section 4.2).

| Screen | Mockup | Addresses which user need? |
|---|---|---|
| Home Page (Morning Theme) | Made without AI in Figma   | Pinned notes, Collections Notes, Whiteboard, and search/filter bar for notes and collections. |
| Note editor – QA block | Made without AI in Figma   | QA blocks on notes for flashcards and quizzes implementation |

| Screen | Mockup | Addresses which user need? |
|---|---|---|
| Collection detail Page | Made without AI in Figma   | Organise notes into collections, and search bar for notes within the collection along with options to add or create new notes. |

## 2.3.Design Rationale

- What did you deliberately leave out / simplify, and why?

**No cloud sync or account system** - All data is local with SQLite; whiteboard collaboration uses peer-to-peer nearby Connections instead of a server or planned nfc, trading multi-device continuity for zero-infrastructure offline use.

**No Room/DataStore -** Persistence is a hand-rolled SQLiteOpenHelper layer with repository classes wrapping raw Cursor/ContentValues, and a single-threaded AppExecutors to avoid fighting SQLite's writer lock. This was a deliberate simplification to keep the threading model predictable rather than introducing full reactive (Flow/LiveData) data layers.

**Whiteboard Picture-in-Picture does not support drawing while in PIP -** Explained in the UX decision we made differently than one AI suggested; this was consciously scoped down rather than solved with a heavier overlay-window rework.

**No live collaborative note editing -** Notes are shared as .quill bundles and imported as new notes with new IDs, so each person has an independent copy. Live collaboration is limited to whiteboards because their immutable, ID-based, append-only strokes are simple to synchronise, while concurrent rich-text editing would require a much more complex conflict-resolution system.

- What's one UX decision you made that an AI assistant suggested differently — and why did you override it (or not)?

  When asked to make the whiteboard drawable while in Picture-in-Picture mode, the AI assistant explained this is impossible with Android's system PIP by platform design (PIP surfaces are touch-inert) and that true interactivity would require abandoning system PIP for a SYSTEM_ALERT_WINDOW overlay, a substantially larger rework. Rather than picking a direction unilaterally, it presented the trade-off and let the team decide. The team chose to keep system PIP with tap-to-expand instead of taking on the overlay-window rework, prioritizing shipping a working feature over a marginal interaction improvement.

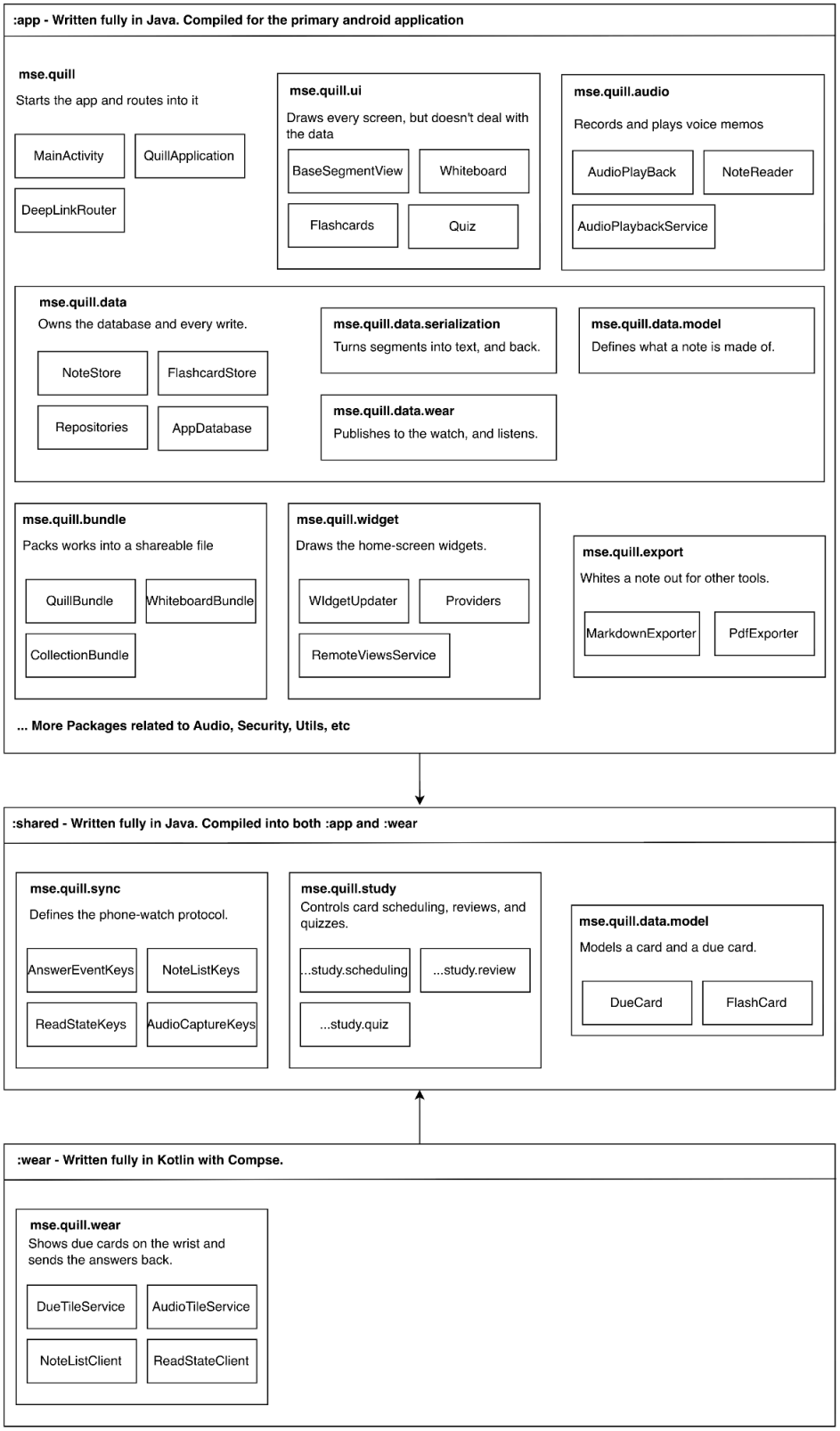
  When in PIP users can see the whiteboard area where the last stroke was made, also changes by others around the area continuously.

# 3. System Architecture Diagram

## 3.1. Architecture Justification

- Why this structure?
- What alternative did you consider, and why did you reject it?
- Walk-through: trace one user action (from 2.1) through every component in the diagram.

## High-Level Modules and Package Overview



The system is three Gradle modules because two of its surfaces run as separate applications on separate devices while sharing one piece of logic that must not diverge. **:app** holds everything that needs Android — the screens, the repositories, the SQLite helper, the widgets are organised into packages whose dependencies run strictly downward, so data/ never imports ui/ or widget/. **:shared** is the one module compiled into both applications. **:wear** then contains no data layer at all: no repository, no database, no cache, because the DataItem published by mse.quill.data.wear is the store, and a second copy could only ever disagree with the first.

The only part of the project where we move away from the “one language, Java” approach is for the WearOS Companion. The original plan was to use Java for as much as possible: the tile and complication can both be built as normal Java Android services, and Kotlin would only be introduced later for the Compose review screen. However, the tile libraries made that difficult. The Material 2.5 library, protolayout-material, provides Java builders, while the Material 3 library, protolayout-material3, is Kotlin-only. Since we wanted the watch tile to follow the same Material 3 design as the rest of the app, using Java would mean falling back to Material 2.5.

The Java :shared module can also be used directly from Kotlin, so there is no major issue with the two languages working together.

| Alternative | Reason for Rejection |
|---|---|
| Java :wear on Material 2.5 tiles | Keeps the project single-language for longer. Rejected because the watch would sit off the app's own design system, and the boundary arrives anyway with the review screen, a postponement that costs a visible inconsistency. |
| Kotlin for :app too | A rewrite of a working Java codebase for no behavioural gain, and it would take a lot of time for regression of already implemented features. |

**Walk-through:** the student answers a due card on their watch

**1. mse.quill.wear** — DueTileService renders the count; the student opens ReviewActivity and taps correct. There is no database here to consult.

**2. mse.quill.sync** — AnswerEventKeys defines the message: card id, correct, and the timestamp it was answered at. That last field exists because an answer given at 08:00 may not arrive until 22:00, and anchoring the interval to arrival would silently push the card fourteen hours further out.

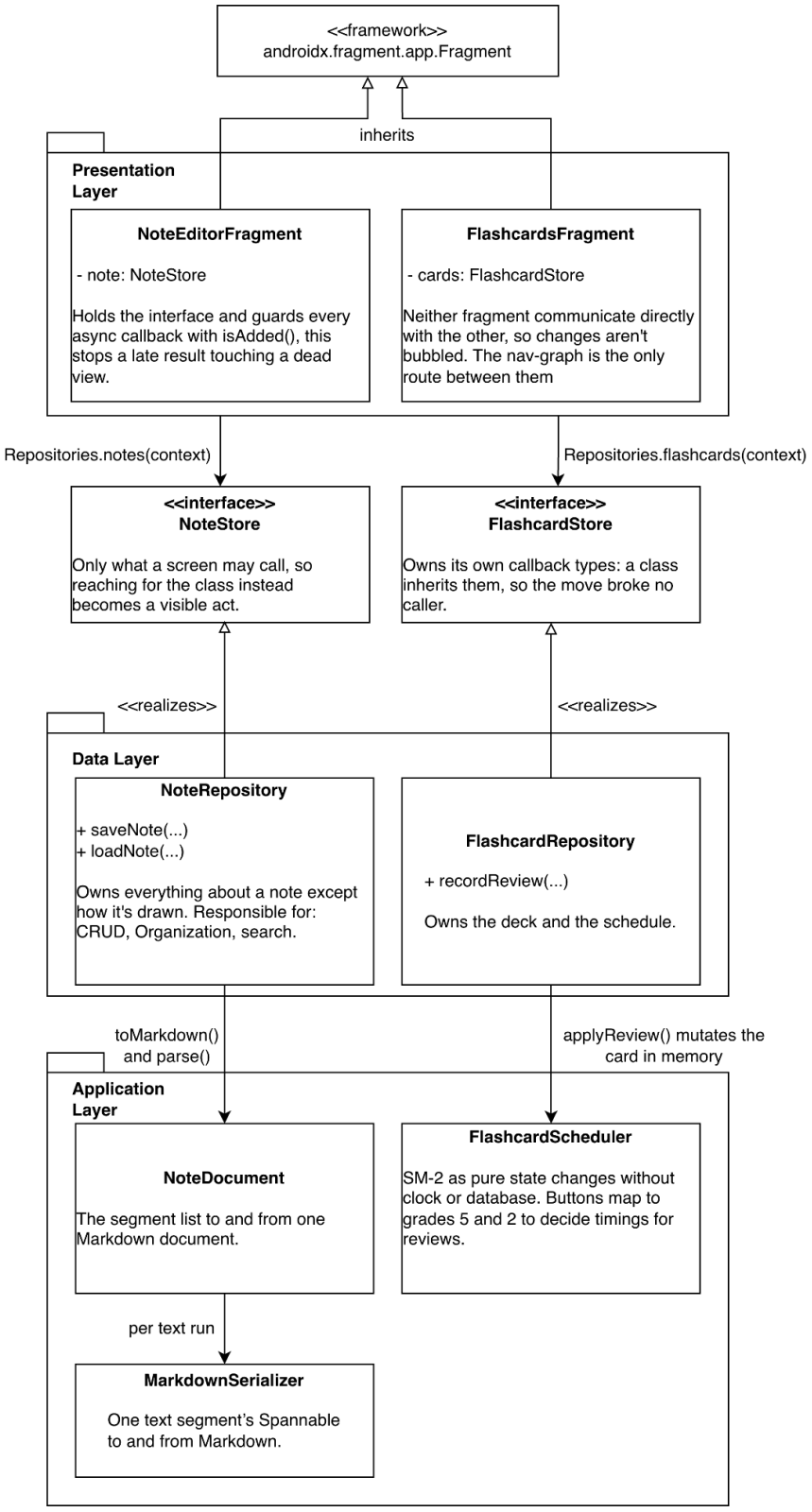
**3. mse.quill.data.wear** — WearAnswerListenerService receives it on the phone; the same *Keys class reads it and writes it, which is why the protocol lives in :shared.

**4. mse.quill.study.scheduling** — FlashcardScheduler.applyReview advances the card: 1 day, then 6, then interval × easiness. Pure state transitions, the identical compiled class the watch itself links.

**5. mse.quill.data** — FlashcardRepository persists the five SM-2 columns on the single disk thread, then announces the change.

**6. mse.quill.widget and mse.quill.data.wear** — both subscribe: the Flashcards widget redraws and the projection republishes, without data/ naming either of them.

## Layered Diagram for Notes and Flashcards



Notes and flashcards are the basis of the app, every other feature is a surface onto one or the other, so this is the slice of the architecture whose structure matters most, and the one the rest of the codebase follows. The diagram shows three layers whose dependencies point one way only: presentation depends on the data boundary, the data layer implements that boundary, and both reach down into pure-logic classes that depend on nothing.

The two screens are drawn side by side because they are deliberately the same shape twice, the repetition is the argument that this is a rule of the codebase and not one screen's arrangement. Each fragment holds *NoteStore* or *FlashcardStore* rather than the concrete repository, so the only methods a screen can reach are the ones meant for it.

Both are obtained from a static factory *(Repositories.notes(context))* rather than constructed, so no fragment ever names a data class directly. The repositories own the SQL, the transactions and the files, and answer through callbacks delivered back on the main thread, which is why every callback is guarded with *isAdded()*: the Handler.post that delivers it cannot be cancelled, so the guard cannot stop a late result arriving, only stop it touching a view that no longer exists. The application layer sits below the data layer rather than between it and the UI, because these classes are called into rather than routed through, and that is precisely what allows them to carry no Android dependency NoteDocument and MarkdownSerializer are separate for the same reason at a smaller scale.

| Alternative | Reason for Rejection |
|---|---|
| MVVM - a ViewModel tier between Presentation and Data | It would remove the *isAdded()* guards for free, since *viewLifecycleOwner* unsubscribes at *onDestroyView*. |
| Service/Manager tier between UI and data | Would have made the diagram conventional. Every method would have been pass-through, because there is no cross-repository orchestration to host, and it would have forced the study logic to depend on the data layer, which is exactly what makes the *:shared* module reusable. |

**Walk-through:** a student writes a Q&A block and later answers the card it produces

**1. androidx.fragment.app.Fragment** — Both screens inherit from it, and its lifecycle is why the guards below exist: Android may destroy either fragment at any point after a call is made and before the answer arrives.

**2. NoteEditorFragment** — The student types a question and answer into a Q&A block during the lecture. 500 ms after the last keystroke the fragment fires its autosave. Nothing was pressed; there is no save button.

**3. NoteStore** — The save goes out through the interface, obtained earlier from Repositories.notes(context). The fragment never names NoteRepository.

**4. NoteRepository** — Realizes that interface and does the work on the single disk thread, inside one transaction: the document, the media asset rows, the whiteboard index and the full-text index all commit or none do.

**5. NoteDocument** — toMarkdown() converts the whole segment list into the single document stored in notes.content_blob. The Q&A block is emitted as a fence carrying its own id, which is the detail everything later depends on.

**6. MarkdownSerializer** — Called by NoteDocument once per text run, turning the question's bold and italic spans into **…** and _…_.

**7. FlashcardStore (from the note editor)** — The student turns the note into flashcard. This is why the editor holds a second interface: generation is initiated where the content is, not on the flashcards screen.

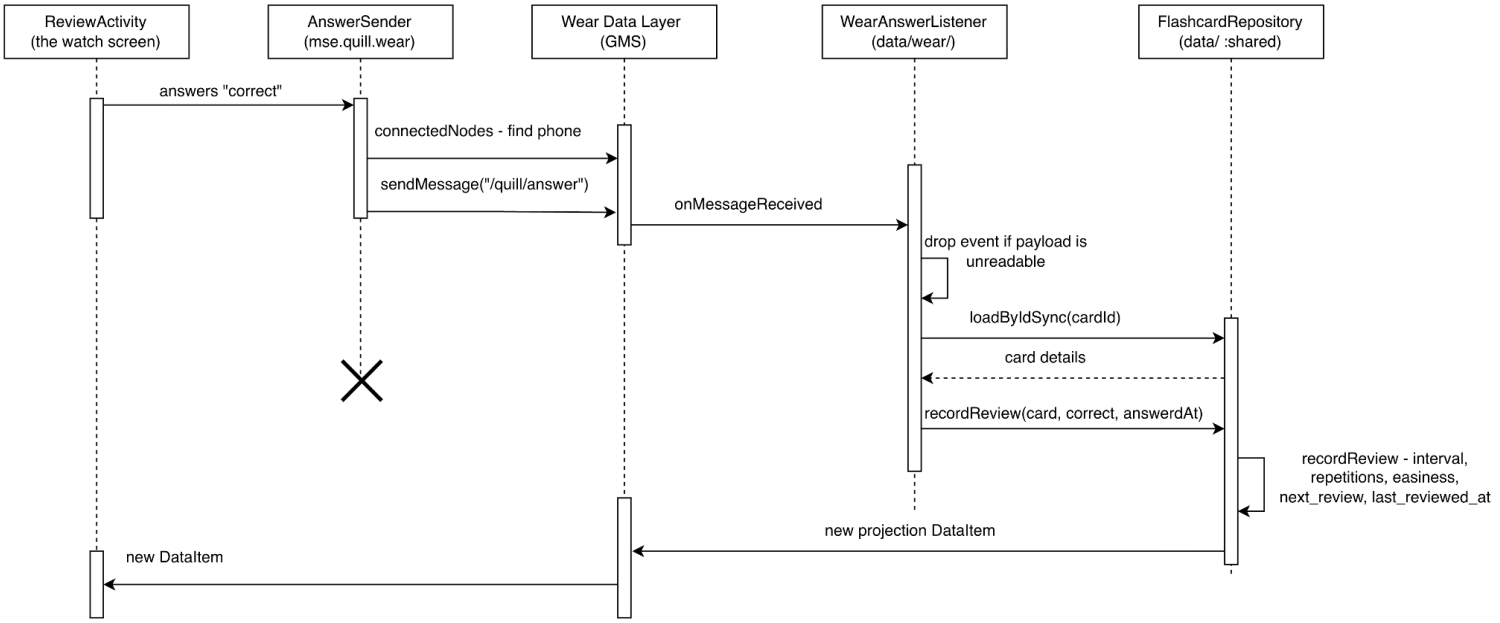
**8. FlashcardRepository** — syncFromNote matches on the fence id and writes only front and back. It never touches the SM-2 columns, so editing a typo later will not discard what is known about recall.

**9. FlashcardsFragment** — The student opens the deck. The fragment guards its load callback with isAdded(), because they may have navigated away while the query was queued.

**10. FlashcardScheduler** — The student answers correctly. applyReview mutates the card in memory: 1 day, then 6, then interval × easiness. Pure state transitions, no clock and no database.

**11. FlashcardRepository (return leg)** — Persists the five resulting columns and notifies, so the widget and the watch projection catch up.

### Sequence Diagram for Interaction between WearOS and Phone for Flashcards




This flow crosses paths between phone and the watch, and carries states that both devices could disagree about. That is why only one device is allowed to hold an opinion about a card's schedule. The watch receives a *DueCard* containing only the card ID, question, answer, due date and title. It deliberately does not receive scheduling data such as easiness or interval, so scheduling always happens on the phone using the shared *FlashcardScheduler* from *:shared*. The watch sends answers back as *MessageClient* events rather than *DataItems* because multiple answers must not overwrite each other. Each event also includes its timestamp, so delayed delivery does not affect the actual review time. After the phone updates the schedule, it republishes the data, automatically updating the watch's tile and complication.

| Alternative | Reason for Rejection |
|---|---|
| Let the watch poll for the new count | Less code than publishing. Rejected because the watch cannot tell a current projection from a stale one by looking, and a surface that finds out on its next poll shows a count that is already wrong |
| Run SM-2 on the watch | Could have been possible since *:shared* is linked into *:wear* and the scheduler is present there. But this gives two devices opinion about the same *easiness* with no conflict resolution. |

**Walk-through:** a student answers a due card on their watch

**1. ReviewActivity —** the student taps correct. The screen advances immediately; it never waits for the network, because a review paced by the radio is not a review.

**2. AnswerSender —** asks the Data Layer for *connectedNodes* to find the phone, then sends to every node returned rather than picking the first.

**3. sendMessage("/quill/answer") —** the payload is three values: card id, correct, and answeredAt. The path and the key names come from *AnswerEventKeys* in *:shared*, so both ends compile against the same constants and neither can drift from the other. The AnswerSender lifeline ends here and it does not wait for a result.

**4. WearAnswerListener (data/wear/) —** *onMessageReceived* runs on a background binder thread, which is already the right kind of thread, so the database read below is a direct blocking call rather than a hop onto the executor. It drops the event outright if the payload is unreadable, if the card id is missing, or if *answeredAt* is absent.

**5. loadByIdSync(cardId) —** returns the card with its SM-2 state, the three fields the watch has never held. If the card was deleted on the phone while the watch held a stale projection naming it, there is nothing to advance and the answer is discarded.

**6. recordReview(card, correct, answeredAt) —** calls *FlashcardScheduler.applyReview* in *:shared*, which mutates the card in memory: one day, then six, then interval × easiness. The repository then persists the five resulting columns,

**7. Republish —** the same method publishes the updated projection as a new *DataItem*. It is ordered after the callback deliberately, so that a review done on the phone advances at the speed of the database write rather than a Data Layer round trip.

**8. Back on the watch —** *ProjectionListenerService* wakes on *onDataChanged*, stores nothing, and simply asks the tile and the complication to redraw. Both read the *DataItem* themselves, so there is no ordering problem: by the time the listener runs, the new value is already in the Data Layer's store.

### 3.2. Third-Party Libraries

:app declares 14 external libraries; all but one are AndroidX, Material Components or Play Services. ZXing is the only non-Google dependency and the only one worth justifying at length

| Library | Purpose | Why this one (over alternatives)? |
|---|---|---|
| com.google.zxing:core | Generation of a QR code containing information about the session such as: session code, version, and token. This is used for collaborative session sharing for the whiteboard. | **Web QR Service:** Quill is marketed as offline-first that can be used without network, so this would defeat it's core value.<br><br>**Developing from scratch:** QR generation is a complex process with multiple stages like mode selection (numeric, byte), block interleaving, bitstream assembly, etc.<br><br>Developing it from scratch was not feasible for the scope of this project, and given how little the QR code generation actually adds to the overall value of the product developed. |

## 4. Developer Diary

This section documents the development process and AI interaction.

### 4.1. Development Milestones

| # | Date | Activity | Result | Issues |
|---|---|---|---|---|
| *1.0* | *2026-06-17* | *Initial Architecture and Database setup* | *Initial Code base and Database setup* | *No consideration for live collab and validation* |
| *1.1* | *2026-06-17* | *Initial Text Editor* | *Basic Text editor* | *Embedding missing for images, audio* |
| *1.2* | *2026-06-19* | *Initial Whiteboard* | *Basic whiteboard* | *Simple whiteboard with no collaboration features* |
| *2.0* | *2026-07-13* | *Note editor keyboard/scroll handling* | *Reverted after ~10 failed AI-driven attempts; kept only unrelated good parts (headings/lists, tap-to-focus)* | *Each fix traded one visible bug for another (toolbar detaching, notes flying off-screen)* |
| *3.0* | *2026-07-15* | *Audio embedding into notes* | *User can record audio into notes* | *Deletion of audio not implemented* |

| # | Date | Activity | Result | Issues |
|---|---|---|---|---|
| *4.0* | *2026-07-22* | *Whiteboard for single user* | *Whiteboard feature done for single user* | *Collaboration not yet implemented to guarantee current features work with multiple users* |
| *5.0* | *2026-07-30* | *Flashcard and Quiz implementations* | *Flashcard decks, review scheduling and quizzes* | *Card and block linking was assumed to be unblocked because segment IDs became stable in July. However, they were only stable within a session, as BaseSegmentView regenerated them on every reload while Q&A stored none.* |
| *5.1* | *2026-08-6* | *Audio waveform block in notes* | *Long-press-to-delete implemented* | *Draft gesture logic would have broken vertical scroll-through; caught and fixed before shipping* |
| *5.2* | *2026-08-08* | *Sharing notes, whiteboard and collections* | *Notes,whiteboard, collections sharable as .zip* | *Imbedded notes not sharable and auto import not implemented yet with .quill filetype* |
| *6.0* | *2026-08-09* | *Sharing completed* | *Whiteboard embedded with notes now also exported, with .quill extension so clicking on it opens it in app* | - |
| *6.1* | *2026-08-12* | *Live whiteboard collaboration (Epic C, Nearby Connections)* | *Two-device sync working (strokes, undo, clear)* | *Joiner crash from whiteboardId foreign-key mismatch on first live test* |
| *7.0* | *2026-08-13* | *Security features* | *Collection can be locked, Quill has biometric lock and unlock* | *Bugs stemmed from locked collections: notes from locked collections being shown in later implementations of Widgets and WearOS* |
| *8.0* | *2026-08-15* | *WearOS implementation* | *Review flashcards, record audio on watch, play aloud feature from watch* | *Lots of issues related to UI since fitting all content on a smaller screen is difficult. Needed multiple sessions explaining UI use-cases to AI.* |
| *9.0* | *2026-08-19* | *Home-screen widgets (Collections, Whiteboards, Flashcards)* | *Three AppWidgetProvider widgets shipped* | *5-bug chain around locked-collection interaction; two bugs found by code reading, not stack traces Fixed at end* |

| # | Date | Activity | Result | Issues |
|---|---|---|---|---|
| *9.1* | *2026-08-20* | *Whiteboard collab stability (fix/whiteboard)* | *Non-host quit crash fixed* | *requireActivity() called from a detached fragment in an async callback* |
| *10* | *2026-08-22* | *Whiteboard for multi user and Ux improvements* | *Whiteboard with more than 2 users, users can exit and join at any time, not just start. Ux improvements with profile, haptics,study streak and welcome messages* | *UI fix required for ratio home screen* |
| *10.3* | *2026-08-23* | *Whiteboard Picture-in-Picture (feat/pip)* | *System PIP wired up, tap-to-expand instead of draw-in-PIP* | *Feature scope negotiated rather than over-engineered; session unverified on real device (no adb access)* |
| *10.4* | *2026-08-28* | *Refactor* | *AppDatabase was split, and the read-aloud controller brought the node editor under 1000 lines.* | *-* |

## 4.2.AI Interaction Log — Critical Incidents

| # | Milestone | What happened | AI's output (brief) | Your action / validation |
|---|---|---|---|---|
| *1* | *2.0 - Note editor keyboard/scroll* | *The editor keyboard covered the active line; auto-scroll didn't compensate. AI iterated ~10 times across different mechanisms (fullScroll, custom cursor-scroll math, WindowInsetsCompat, adjustResize/adjustPan), each attempt fixing one visible symptom while breaking another (toolbar detaching, short notes flying off-screen, title scrolled out of view).* | *Repeated code patches per mechanism, no working end state reached.* | *Rejected the entire line of attempts and reverted all scroll-related changes, keeping only the unrelated good parts (headings/lists, tap-to-focus). Lesson recorded: ask for a differentiated reproduction earlier and stop after 2-3 failed attempts at the same mechanism.* |
| *2* | *5.1 - Audio waveform delete gesture* | *Long-pressing an audio waveform block to delete it instead scrubbed/played the clip.* | *First draft treated any drag exceeding touch slop as a scrub which would have swallowed vertical scroll gestures whenever a recording sat under the finger.* | *Caught during the AI's own code review before shipping (not via a bug report); fixed with a per-direction gesture split so vertical drags return false and pass through to the parent scroll view. Verified on the emulator by hand.* |

| # | Milestone | What happened | AI's output (brief) | Your action / validation |
|---|---|---|---|---|
| 3 | *5.2 - NFC + Wi-Fi Direct replaced by Nearby Connections and a QR token* | *Collaboration had been scoped from the start around an "NFC tap-to-pair handshake" over Wi-Fi Direct. After a mid-term presentation suggestion that Quick Share might be the better route, we asked the AI to review whether that original basis was still right before any of it was built.* | *The review invalidated most of the plan. The NFC step was specified on a dead API, "tap-to-pair" meant Android Beam / NDEF push, deprecated in Android 10 and non-functional on modern devices, since two phones cannot push to each other* | *Rewrote the feature specification and took suggestions from the AI regarding a QR approach. It also made sense from a UX perspective since QR can be scanned from across the table rather than having to literally tap the phone to pair.* |
| 4 | *6.1 - Live whiteboard collaboration crash* | *First live two-device test: the joiner app silently crashed when pressed back to the home screen; the host only saw a misleading "session ended" message.* | *AI diagnosed the true cause as a SQLiteConstraintException : incoming strokes kept the sender's whiteboardId, violating a local foreign-key constraint since host and joiner have independent local board rows.* | *Applied the AI's fix (re-tag incoming items onto the local whiteboardId at all three receive sites, deliberately leaving authorId untouched) and verified sync, undo, and clear across both devices.* |
| 5 | *8.0 - WearOS stack decision* | *Our plan was Java-first: build the tile and complication as Java Android services and introduce Kotlin only later for the Compose review screen. Before starting we asked the AI to confirm the tile libraries supported that.* | *AI's Research showed protolayout-material3 (Material 3 Expressive, MaterialScope, Material3TileService) is Kotlin-only with no Java builders, while only the Material 2.5 protolayout-material exposes them — so a Java tile is necessarily a Material 2.5 tile.* | *Reversed the decision mid-discussion and made :wear Kotlin from the first commit, keeping :app and :shared in Java so the boundary sits at the module edge.* |
| 6 | *9.0 - Home-screen widgets + collection locking* | *After locking a collection, a pinned note stayed visible in its widget and tapping it deleted the real note; creating a note in a locked collection then crashed the app on every subsequent launch.* | *First fix (exception guards in the widget's RemoteViewsFactory) did not stop the crash: the real second cause was unguarded Wear OS publisher calls in MainActivity.onCreate, a genuinely separate bug rather than a failed first fix.* | *Diagnosed and fixed sequentially across five rounds; two further bugs (in NoteRepository/NoteEditorFragment) were found by manual code reading rather than from a stack trace.* |
| 7 | *10.3 - Whiteboard Picture-in-Picture scope* | *Asked for the whiteboard to be drawable while in PIP mode.* | *AI explained this is impossible with Android's system PIP by platform design (PIP surfaces are touch-inert), and that real interactivity would require replacing system PIP with a* | *Chose to keep system PIP with tap-to-expand rather than take on the overlay rework.* |

| # | Milestone | What happened | AI's output (brief) | Your action / validation |
|---|---|---|---|---|
| | | | *SYSTEM_ALERT_WINDOW overlay : a much larger rework, then presented that trade-off instead of deciding unilaterally. Separately caught a FLAG_SECURE-causes-black-PIP-thumbnail interaction with the app's biometric lock before shipping.* | |

***Note**: A more detailed conversation log can be found in memory/conversation.md of the repository where we've logged most of the interactions that took place between us and the AI. The AI was explicitly prompted to update the conversation file after the end of each session.*

# 5. Android Features Report

## 5.1. Features Used

List every relevant Android platform feature your app relies on (e.g., sensors, camera, location, local storage, notifications, permissions, lifecycle-aware components, etc.).

| Feature | Where it's used (screen/class) | Why this feature was needed |
|---|---|---|
| *Foreground Service (FOREGROUND_SERVICE_MEDIA_PLAYBACK)* | *AudioPlaybackService* | *Keeps voice-memo/read-aloud playback alive with lock-screen/notification controls when the app isn't in the foreground* |
| *App Widgets (AppWidgetProvider + RemoteViewsService)* | *Three home-screen widgets: Collections, Whiteboards, Flashcards* | *Quick access to pinned notes/whiteboards/due flashcards without opening the app* |
| *Nearby Connections (Bluetooth/BLE + Wi-Fi permissions)* | *Whiteboard collaboration session join/host flow* | *Peer-to-peer live collaboration without a server, consistent with the app's offline-first design* |
| *Biometric authentication (androidx.biometric)* | *Private/locked collections, and lock on the entire application.* | *Protects sensitive notes and/or application (e.g. exam drafts) behind fingerprint/face auth* |
| *Wear OS companion (Data Layer: MessageClient/DataClient)* | *: wear module: Capture, ReadAloud, Review activities; Tile services; Complication service* | *Voice capture, audio playback for notes and flashcard review from the wrist without pulling out the phone* |
| *WorkManager* | *Background/scheduled jobs (e.g. flashcard due reminders)* | *Guarantees deferred work survives process death and respects constraints* |
| *Custom URI/MIME deep links (singleTask MainActivity)* | *.quill/.quillboard/.quillpack file types, quill://whiteboard/join scheme* | *Import/export of notebooks and QR-code-based whiteboard session joining* |
| *Local persistence via SQLiteOpenHelper* | *Repository classes across the app* | *Offline-first data storage without a Room dependency, single-threaded* |

| Feature | Where it's used (screen/class) | Why this feature was needed |
|---|---|---|
| | | *AppExecutors to avoid fighting SQLite's writer lock* |
| *Notifications (POST_NOTIFICATIONS)* | *Playback controls, flashcard reminders* | *User-visible controls/reminders outside the app UI* |
| *Picture-in-Picture (PictureInPictureParams)* | *MainActivity.java, WhiteboardFragment* | *Used during whiteboard collaboration because it's useful to see what the other people are drawing even when you're using something else on the phone.* |
| *Camera capture (ACTION_IMAGE_CAPTURE + FileProvider)* | *ImageEmbedder* | *Photographing a slide or a board straight into a note without leaving the editor* |
| *Audio recording (MediaRecorder)* | *AudioRecorder, WaveformView, WaveformCache, MemoRecorder (watch)* | *Voice memos embedded as note segments, recordable from phone or watch* |
| *Text-to-speech (TextToSpeech)* | *NoteReader, WearReadListenerService, WearReadControlListenerService. ReadAloudActivity as the watch control surface.* | *Reading a note aloud on the phone, also controllable from the watch* |
| *ML Kit code scanner (GmsBarcodeScanning)* | *SessionScanner* | *Scans the collaboration QR in the Play services process for whiteboard.* |
| *PDF Generation (PdfDocument)* | *PdfExporter* | *Exporting a note as PDF* |
| *Haptic feedback (HapticFeedbackConstants)* | *Haptics* | *Haptics for deleting contents, and flashcard reviews.* |
| *Stylus input (MotionEvent TOOL_TYPE getPressure)* | *WhiteboardView* | *Pressure-sensitive drawing, and telling a stylus from a finger so palm contact doesn't accidentally draw* |
| *Window Insets (WindowInsetsCompat)* | *6 classes centered on mse.quill.ui.common* | *Edge-to-edge layout with the status bar taking the color of the screen beneath it for visual consistency.* |
| *MediaStore / scoped storage* | *NoteExportStore, ImageExporter, StoragePermission, WhiteboardExportController* | *Writing exports to a shared storage in the device.* |

**Note on location permissions:** Quill does not use location features or any location APIs. The location permissions are declared only because Google Nearby Connections requires them for Bluetooth and Wi-Fi discovery on older Android versions. ACCESS_COARSE_LOCATION is limited to API 28 and below, while ACCESS_FINE_LOCATION is limited to API 32 and below. This prevents newer devices from requesting permissions they do not need.

The Bluetooth scanning permissions use usesPermissionFlags="neverForLocation" to explicitly indicate that scan results are not being used to determine the user's location. There is no LocationManager or FusedLocationProviderClient usage in the codebase. The unused location_lat, location_lng and location_name database columns are only reserved for the planned geotagged-notes feature.

## 5.2.Implementation Details

- For at least three outstanding features above, describe how you implemented it (key classes/APIs involved, lifecycle considerations, permission handling, etc.) in enough detail that someone could understand your approach without reading the code.

### (a) Nearby Connections whiteboard collaboration (mse.quill.collab)

Two students can share a whiteboard directly between their phones using Google Nearby Connections, without a server. *CollabSession* handles both roles: the host uses *startAdvertising()* and the joiner uses *startDiscovery()*. Both use *Strategy.P2P_STAR*, which is designed for one central device communicating directly with multiple nearby devices, making it suitable for one student hosting a shared board.

A random session token is shown as a QR code. Nearby's endpointId cannot be used for this because it is generated locally on each device, so the host would have no usable ID to show. After the joiner scans the token and finds the matching device, *requestConnection()* establishes the connection.

Whiteboard changes are sent as JSON using *Payload.fromBytes()* and handled in *onPayloadReceived().* The host first sends a complete snapshot from the database so the new device starts with the current board, then sends individual changes as they happen. Strokes have unique IDs, so receiving the same stroke twice is harmless.

*CollabSessionHolder* keeps the connection alive when navigating away from the whiteboard, while *WhiteboardCollabController* checks *fragment.isAdded()* before updating the UI because Nearby callbacks can arrive after the Fragment has been detached. *CollabPermissions* handles the Bluetooth, Wi-Fi and location permissions required on different Android API levels, and a 20-second timeout prevents discovery or connection from hanging indefinitely.

### (b) Home-screen widgets (mse.quill.widget)

Quill provides three home-screen widgets: Collections, Whiteboards and Flashcards. Each displays a scrollable list, with rows opening the corresponding screen when tapped.

Widgets are rendered by the launcher, not by Quill, so they use Android's *RemoteViews* system. Each *AppWidgetProvider* creates the widget and connects it to a *RemoteViewsService*, whose *RemoteViewsFactory* loads the list data in *onDataSetChanged()*. Because this method runs synchronously, the repositories provide separate blocking methods such as *loadDecksForWidgetSync()* rather than adding them to the normal screen-facing interfaces.

Widgets are refreshed through *DataChangeNotifier* and *WidgetUpdater* instead of polling. The notifier runs from *QuillApplication* because widgets can exist even when no Activity is running. When data changes, only the affected widget list is refreshed using *notifyAppWidgetViewDataChanged().*

For taps, each widget uses a *PendingIntent* template. Individual rows add their own ID through *setOnClickFillInIntent()*, allowing the same template to open different collections or notes. The template must be mutable because the system needs to add this row-specific data when the user taps.

Locked collections are filtered out before the widget data is created, so protected content never appears on the home screen. If a protected deep link is tapped, *DeepLinkRouter* waits for the biometric unlock and then continues the original navigation.

### (c) AudioPlaybackService (mse.quill.audio)

Quill separates two types of audio: voice recordings, which can be played and seeked, and read-aloud, which plays a note from beginning to end using both text-to-speech and embedded recordings. *AudioPlayback* handles recordings, while *ReadAloud* acts as a sequencer using *NoteReader* for text and *ClipReader* for recordings. Both use Android *AudioFocus* so playback behaves correctly when another app needs the audio output.

*AudioPlaybackService* keeps recordings playing when the app is no longer visible. It runs as a foreground media service and exposes a *MediaSession* for play, pause, stop and seek controls. The actual playback state remains in *AudioPlayback*, so the service itself does not need to maintain a separate copy of the state.

Read-aloud can also be controlled from a Wear OS watch through the Wear Data Layer. *WearReadListenerService* receives a note ID and starts the reading on the phone, keeping the note content and text-to-speech engine on the phone. *WearReadStatePublisher* sends the current reading state back to the watch, while *WearReadControlListenerService* forwards play/pause controls to the same ReadAloud logic used by the phone. State is sent only when needed and is throttled to avoid excessive updates.

*TextToSpeech* initialisation is asynchronous, so ReadAloud checks whether it has been shut down before using the engine. This prevents callbacks arriving after the user has left the screen from causing crashes. Recording requires *RECORD_AUDIO* at runtime; playback itself requires no permission, although Android 13+ may require notification permission for the foreground-service controls to appear.

- Did you handle edge cases (e.g., permission denied, sensor unavailable, no network, configuration changes)? How?

a) **Nearby permission denial:** CollabPermissions.missing(...) is checked before starting or joining a session, so missing Bluetooth/Nearby permissions are detected before the Nearby Connections API is called. The required set is selected according to the Android API level.

b) **Offline operation:** No special network-error handling is needed because Quill is designed as an offline-first application. Local SQLite storage is used for the main data, while Nearby Connections provides local collaboration.

c) **Fragment detached during an asynchronous callback:** In Incident 3 (section 4.2), a requireActivity() call inside a Nearby Connections callback could run after the user had already left the fragment. That caused a crash because the fragment was detached by then. The callback now checks that the fragment is still attached before accessing its activity.

d) **Widget data-load crashes:** In Incident 4 (section 4.2), an exception inside onDataSetChanged() ran on the binder thread and could crash the entire app. The

locked-collection case is now filtered out when the widget data is queried instead of relying only on a later try/catch.

e) **Fast Navigation:** Opening and leaving a note before its asynchronous load returned once made autosave read the empty fields as "the user emptied this note" and delete it. A contentLoaded flag makes autosave a no-op until the read lands, so empty fields mean "not read yet" rather than "emptied".

f) **Untrusted import from another device:** A .quill bundle can arrive from another device without any trusted sender identity, so it is treated as untrusted input. Entry names are restricted to plain files directly inside media/; checking only for .. is not sufficient, since paths such as media/../../databases/quill.db could otherwise escape the intended directory. The 256 MB limit is also enforced while bytes are being read rather than relying on the declared archive size, protecting against zip bombs. If one note in a collection bundle is corrupted, it is skipped instead of failing the entire import, and the result reports how many of the total notes were imported.

g) **Dangling references after deletion:** Deleting a whiteboard does not modify notes that previously referenced it, since changing the user's note would be more intrusive than leaving the reference. Instead, the editor displays “This whiteboard was deleted”. Similarly, if an embedded image or audio asset no longer exists, the embed is removed when the document is parsed and the surrounding text is joined together, preserving the document structure.

## 5.3.AI’s Role in Android-Specific Code

- Did AI suggest correct usage of these Android APIs, or did you need to correct/research further?

  AI assistance was mostly reliable when choosing the Android APIs themselves. For example, the suggestions to use Nearby Connections for peer-to-peer communication, WorkManager for background tasks, and androidx.biometric for locking were appropriate.

  The bigger problems appeared around lifecycle behaviour and debugging. One example was the use of requireActivity() inside an asynchronous Nearby Connections callback. That worked while the fragment was attached but failed once the user had already navigated away (Incident 5, section 4.2). There was also a widget fix that solved one cause of the crash while leaving a second independent cause untouched (Incident 4).

- Were there any Android-specific hallucinations (e.g. deprecated APIs, incorrect lifecycle assumptions, wrong permission models)?

  No clearly deprecated API hallucinations were found in the material reviewed. The more consistent issue was that an AI-generated fix often addressed the reported symptom without first checking whether another root cause could produce the same failure.

# 6. Lessons Learned

- **Technical insights:**

  The project uses a hand-written SQLiteOpenHelper layer together with a single-threaded AppExecutors. This made the threading model fairly easy to follow. There was no reactive query layer to deal with, and concurrent writes were controlled rather than being left to compete for SQLite's writer lock. The downside is the amount of repository, Cursor, and ContentValues code that has to be maintained manually. For the size of this project, the trade-off was reasonable. If the schema grows much further, Room would probably be worth reconsidering.

  Keep PR as small as possible. We had a discussion about file,variable naming scheme but didn't discuss commit messages.

  Be ready to adapt to changes on requirements as while developing, you can notice something is out of scope, some of it doesn't work for example during mid-term presentation we got a suggestion to use quickshare instead of wifi-direct which we used in the end.

  We thought live notes collaboration would be possible within the time frame but after discussion, we realised it takes a lot of changes to current architecture considering the tracking of 2 cursors, changes with just offline sync.

  Verify wiring in the built artifact when the system UI won't cooperate. Confirming the Wear tile previews on screen proved impossible. For example, the carousel is not reliably drivable by adb, the tile editor auto-dismisses, and input rotary encoder scroll pulled down the notification shade instead. What worked was aapt2 dump resources and aapt2 dump xmltree to prove the PNGs were packaged and the services pointed at them, and leaving the visual check to the team.

- **Architectural insights:**

  Keeping :shared as a plain Java library with no Android dependencies worked better than expected. The SM-2 scheduling and quiz-generation code is therefore the same on the phone and watch, and it can be unit-tested on the JVM without an emulator. There is some duplication around the module boundary because each platform still handles its own persistence and UI, but that duplication is relatively small compared with the benefit of keeping the shared logic independent of Android.

  A deferred cost is acceptable as long as the decision to defer it is documented. When we extracted the shared module, we deliberately kept the existing package names unchanged. This meant the move affected eleven files but required no import changes, making the change easy to review at a glance. Two weeks later, we carried out the cleaner package rename, which required adding seventeen imports across two modules. That was the cost of the earlier decision. Because the trade-off had been documented rather than left as an oversight, addressing it later was a planned task rather than an unexpected problem.

- **UX insights:**

  **Accepting Tradeoffs:** The Picture-in-Picture decision was a good example of accepting a platform limitation instead of building a complicated workaround. System PIP surfaces are not designed for arbitrary touch interaction, so adding a SYSTEM_ALERT_WINDOW overlay just to make the PIP view interactive would have introduced more complexity. The team chose tap-to-expand instead, which was a more practical use of the remaining development time.

**Focus on Learnability:** Having a user-onboarding step with a starter template rather than throwing users directly into the home-screen was a good move in terms of learnability. On top of that, having multiple welcome messages makes the whole experience feel less repetitive and more engaging.

**Experiment with AI in UX Design:** The initial UI mockups were designed entirely by the team at the beginning of the project. However, due to time constraints, not all screens could be designed in advance. During implementation, we therefore allowed the AI to take responsibility for designing the remaining UI, using the existing mockups as a reference for the overall visual direction.

This worked reasonably well, but it also resulted in several rounds of revision, as the AI-generated designs sometimes required additional prompting and critique to better match our expectations. In retrospect, we could have used AI earlier in the design phase to generate initial mockups for the most important screens. This would likely have established a more complete design direction from the outset and reduced some of the back-and-forth during implementation.

- **AI-related insights:**

  When an AI-generated fix does not solve a crash, the problem may not be that the first fix was simply wrong. There can be a second, unrelated root cause, as happened with Incident 4 (section 4.2). Incident 1 also showed that repeatedly trying variations of the same mechanism is not especially productive. After two or three unsuccessful attempts, it is usually better to step back and collect a more precise reproduction case.

  Sometimes you need to prompt differently to get a solution, we realised at some time to start adding instructions to do stuff accordingly, also to have a conversation to confirm on changes before committing to implementation

  AI should be given as much context as possible about the entire project from the outset. This allows it to reason about the overall architecture and make appropriate design decisions, rather than simply implementing the immediate requirements stated in a prompt.

  We realized this after the first few weeks of development, when the AI tended to focus narrowly on solving the problem described in each individual prompt. As a result, architectural concerns were sometimes overlooked. For example: services were not properly separated and functionality was unnecessarily accumulated into single files. To address this, we introduced requirements and architectural documentation directly into the repository through memory files, giving the AI a persistent source of context about the project's overall structure, goals, and design decisions.

- **What would you do again or differently?**

  Have weekly meetings to discuss current ongoing progress within the team and not just see commits of what's happening, sometimes one of the other members was out of the loop as the other was implemented.

  Start documentation from the start – as stated earlier, we implemented a memory folder for AI but not from start which would have benefited us.

  The note-editor keyboard/scroll issue would be handled differently, though. A precise reproduction should have been established before the first AI-assisted fix. Instead, the team spent roughly ten iterations testing slightly different explanations, when the more useful first step would have been identifying exactly which lines and view heights reproduced the problem.

  Trust the code over the notes, and date the notes. Our **note.md** stated that database migrations were still destructive and that nothing was querying the full-text index, but both statements had

become outdated weeks earlier. We only discovered this when preparing the Future Work section: two items we assumed were still outstanding had already been completed.

This highlighted an important limitation of our documentation process. Even though we had a system for tracking progress and recording remaining tasks, documentation can still become stale when the implementation evolves faster than the notes are updated. This is particularly important for onboarding documentation, since outdated information can be repeatedly relied upon and propagated to new contributors. Documentation should therefore be treated as a snapshot of the project at a particular point in time, with implementation and commit history serving as the ultimate source of truth.

# 7. Future Work

- **Possible refactoring:**
  - NoteRepository is over a 1000 lines of code, This can be moved down a layer.
  - A substitutable test seam. This is written into R6 as its honest limit: Repositories is a static factory and fragments call it internally, so a fake still cannot be injected — repository tests remain in androidTest and need a device. The interfaces exist; only the wiring doesn't. It's the single refactor with the clearest payoff, because it converts device-bound tests into JVM tests.
  - WhiteboardThumbnails package sits in ui/whiteboard/ while reading the database and writing into widget cache, so it is neither a screen, nor a view, but still needs decisions about thumbnails

- **Possible extensions:**
  - Multi-device continuity for the whiteboard beyond the current P2P-only session model (e.g. an optional relay for boards that outlive a single Nearby session)
  - Live collaboration for notes: Currently live collaboration is only allowed on whiteboards. Having the same feature on notes would make the app more complete.
  - Whiteboard Host Migration: In the current iteration, when the host of the whiteboard leaves, the session is automatically terminated. In such a case, a different host could be automatically chosen.
  - Transcribe voice memos: When exporting notes to pdf/markdown, the voice memos are pretty much lost. Auto-transcription of voice memos would be useful in that case.
  - Recognize handwriting on whiteboards: Currently even if text block is available, turning handwritten notes digital would be an incredibly useful extension.
  - Support for desktop devices through native applications or web interface: In order to build a “complete” productivity app, the use case of a laptop of desktop computers cannot be denied.